AI Food & Macro Tracker — V1 Design Baseline

Oct 28, 2026 ·

This is the single design baseline for Version 1. It replaces the Design Document v1.1 (23 Oct), the Architecture and Data Models file (24 Oct) and the design parts of the Implementation Guide (24 Oct). Where any earlier file disagrees with this one, this one wins. Every change from those files is listed, with its reason, in the **Discrepancy log** (Appendix A).

Document control

Item	Value
Document	Software Design Document (SDD), V1 baseline
Version	2.0 (draft for review)
SDLC phase	Design (high-level and low-level design)
Delivery model	Agile, 2-week sprints; this baseline is the Sprint 0 output
Builders	Vishal, Riya
Supersedes	Overview v1.0, Design v1.1, Architecture and Data Models v1, Implementation Guide v1 (design content)
Change rule	Any change to a Must requirement, the data model or the API contract needs a new minor version and a line in the Discrepancy log

Document set

Part	Holds	Standard it follows
Sections 1 to 15	Requirements baseline, architecture, data, interfaces, modules, UI, security, tests, operations	IEEE 1016 design views, C4 architecture levels

Part	Holds	Standard it follows
Appendix A	Discrepancy log: every contradiction found in the old files and how it was resolved	Decision log / ADR summary
Appendix B	Product strategy: market research, positioning, what makes this app different	Product brief
Appendix C	Roadmap and delivery: V1 to V3, sprint plan, ownership, Definition of Done	Agile release plan

1. Introduction

V1 lets a user photograph a meal, get an AI estimate of calories and macros, correct it, save it, and see today's progress against a target. Everything else waits for later versions.

1.1 Scope of V1

ID	Feature	In one line
F1	Account and targets	Sign up, stay logged in, enter body details, get calorie and macro targets
F2	Meal photo analysis	Photo goes to a server function, which asks a vision AI model for items and nutrition
F3	Review and log	User corrects items and portions, then saves; manual add when a scan is not possible
F4	Daily scan quota	A fixed number of AI scans per account per day, always visible
F5	Today, history and warnings	Today's progress, a calorie chart, entries kept on the phone for 1, 3 or 6 months

1.2 Out of scope for V1

Barcode scanning, saved meals and custom foods, cloud sync and backup, payments, widgets, iOS, micronutrients, exports, social features, threshold push notifications, and the on-device live-camera model. The **Roadmap and delivery** plan (Appendix C) places each of these in a later version.

1.3 Audience

Vishal and Riya (build), reviewers and testers, and anyone who later extends the app. Readers are assumed to be new to Flutter and Firebase, so terms are spelled out.

1.4 Definitions

Term	Meaning here
Scan	One successful AI analysis of one photo
Scan day	The quota day: midnight to midnight India time (IST), decided by the server
Calorie day	The logging day: midnight to midnight on the phone's own clock
Entry	One food item saved to the log (one row in `meal_entry`)
Meal	All entries saved together from one scan or one manual add
Estimate	Any number produced by the AI model; always labelled as such
Provider	An outside AI API (Google Gemini or xAI Grok)
Must / Should / Could / Won't	MoSCoW priority. Must = V1 does not ship without it

2. Product context and design goals

The design optimises one thing above all: the seconds it takes to log a meal. Calorie apps rarely fail on features; they fail because people stop logging in the first two weeks (see the **Product strategy** (Appendix B)). Every design choice below is judged against that.

2.1 Target user for V1

Indian gym-goers and fitness beginners, 18 to 35, Android, who eat mostly home-cooked and mess or canteen food, want to hit a protein target, and have quit at least one calorie app before.

2.2 Design goals, in priority order

#	Goal	Measured by	V1 target
G1	Fastest logging in the category	Time from app open to meal saved	Median under 15 s for a scanned meal
G2	Trustworthy numbers	Calorie error on the 30-meal test set; share of AI items the user edits	Most single dishes within 25%; edit rate tracked
G3	Never lose or corrupt a log	Critical defects	Zero open at release
G4	Private by design	Meal data and photos on servers	None stored
G5	Cheap to run while free	AI cost per active user per month	Within the budget alert set in section 14
G6	Easy to extend	New AI provider or on-device model added without UI changes	Provider and estimator interfaces in place

2.3 Design principles

- **Today first.** The app opens on today's progress; scan is one tap away.
- **Honest numbers.** AI output is always called an estimate and is always editable.
- **Protein before guilt.** Protein progress gets equal space to calories; warnings are calm and come with one suggestion.
- **Thin client logic, no thin trust.** All calculations run on the phone; only the AI call and the quota live on the server.
- **Ship small, measure, improve.** Anything not needed to log a meal reliably waits for a later sprint.

3. Requirements baseline

V1 has 28 functional requirements (25 Must, 1 Should, 2 Could) and 9 non-functional ones. IDs from Design v1.1 are kept where the meaning is unchanged; a "Changed" mark points to the Discrepancy log.

3.1 Functional requirements

ID	Requirement	Feature	Priority	Changed
FR-01	Sign up with name, email and password (min. 8 characters)	F1	Must	
FR-02	Log in and log out	F1	Must	
FR-03	Session persists across restarts until logout, and works offline after first login	F1	Must	
FR-04	Password reset by email	F1	Must	Yes (was Should)
FR-05	Google sign-in	F1	Could (V1.1)	Yes
FR-06	Profile setup after first login: goal, age, sex for calculation, height, weight, activity; ranges age 16 to 90, height 100 to 230 cm, weight 30 to 250 kg	F1	Must	
FR-07	Calculate calorie and macro targets; user can override; changing the profile recalculates targets unless they are custom	F1	Must	Yes
FR-08	Not-medical-advice note at setup and in Settings	F1	Must	
FR-09	Camera screen with permission explanation, shutter and gallery fallback	F2	Must	
FR-10	Compress photo to long side 1024 px, JPEG quality 75, before upload; delete the temporary file after the call	F2	Must	Yes

ID	Requirement	Feature	Priority	Changed
FR-11	Server function returns structured items and nutrition from a vision AI provider, with automatic fallback to a second provider	F2	Must	Yes
FR-12	Loading state; every failure offers Retry and Add manually	F2	Must	
FR-13	Result lists each item with name, portion, grams, kcal, protein, carbs, fat and an Estimate label	F3	Must	
FR-14	Rename, change grams or remove an item; values rescale proportionally	F3	Must	
FR-15	One tap saves the whole result as one meal	F3	Must	
FR-16	Manual add (name, kcal, optional protein, carbs, fat)	F3	Must	Yes (was Should)
FR-17	Server enforces the daily scan quota per account	F4	Must	
FR-18	Scans left shown on Home and Camera before scanning, and a clear message at zero	F4	Must	Yes
FR-19	Today's totals are the sum of today's entries; a new calorie day starts at zero	F5	Must	
FR-20	Home shows calories eaten and left, and protein, carbs, fat against targets	F5	Must	
FR-21	Edit or delete any entry; totals update at once	F5	Must	
FR-22	Retention of 1, 3 or 6 months (default 3); older entries deleted automatically	F5	Must	

ID	Requirement	Feature	Priority	Changed
FR-23	History chart of daily calories with a target line: 7 days, 30 days, All	F5	Must	
FR-24	Tap a day to see its entries and totals	F5	Must	
FR-25	In-app amber state at 80% of target and red state above 100%	F5	Must	
FR-26	Daily logging reminders at user-chosen times, on by default, can be turned off	F5	Should	Yes (replaces threshold notifications)
FR-27	Evening protein reminder	F5	Could (V1.1)	Yes
FR-28	Delete account in-app and through a public web link; deletes server counters and local data	F1	Must	Yes (was Should)

3.2 Non-functional requirements

ID	Requirement	Changed
NFR-01	Scan returns within 6 s for 90% of requests on 4G; hard timeout 20 s	Yes
NFR-02	Play download size at most 25 MB	
NFR-03	Everything except sign-up, first login, scan and account deletion works offline	
NFR-04	Android 10 (API 29) to Android 15, including 3 GB RAM phones	Yes
NFR-05	HTTPS only; no secret keys in the app or repository	
NFR-06	Photos never stored on a server or kept on the phone	
NFR-07	Contrast at least 4.5:1, tap targets at least 48 dp, works at 130% font scale	Yes
NFR-08	Cold start to Home under 2 s on a mid-range phone	New

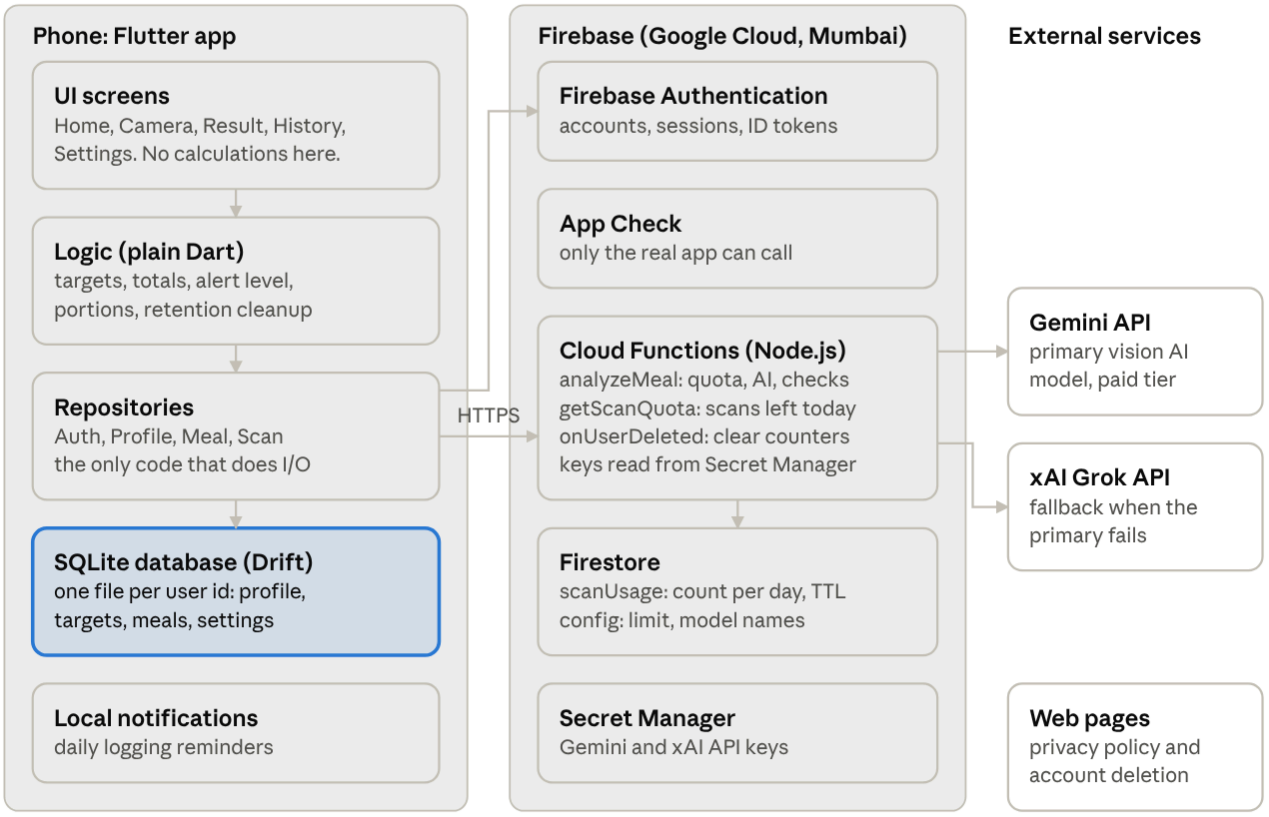
ID	Requirement	Changed
NFR-09	AI spend capped by quota, App Check and a billing budget alert	New

4. Architecture

V1 is a two-tier system: the Flutter app owns all user data and calculations, and Firebase owns identity, the scan quota and the AI call. The phone talks to the cloud for only four things: sign-in, scanning, reading scans left, and deleting the account.

Meal data stays on the phone; the cloud only checks quota and calls the AI

Container view (C4 level 2), Version 1



Arrows are calls. Firestore rules deny all client access; only the functions read or write it.

Container view · phone, Firebase, external AI providers

The highlighted box is the only place meal history lives. The server never sees what a user ate, only how many scans they used today.

4.1 Containers and responsibilities

Container	Owns	Never does
Flutter app	All screens, target maths, totals, alert level, history, chart, retention, reminders, local database	Hold API keys; talk to Firestore
Firebase Authentication	Accounts, password hashes, sessions, ID tokens	Store profile or meal data
App Check	Proves calls come from the genuine app (Play Integrity)	
Cloud Functions	Quota check, provider calls, response validation, counter cleanup	Store or log photos
Firestore	Daily scan counters, config (limit, model names, prompt version)	Accept any direct client read or write
Secret Manager	Provider API keys	
Gemini / xAI	Turn a photo into items and nutrition estimates	Receive any user id, email or history
Web pages (Firebase Hosting)	Privacy policy, account-deletion instructions and request form	

4.2 Technology stack

Layer	Choice
App	Flutter (stable channel), Dart, Android first
State and navigation	Riverpod (manual providers, no code generation), go_router
Local database	Drift on SQLite
Camera and images	camera, image_picker, flutter_image_compress
Charts	fl_chart
Notifications	flutter_local_notifications
Firebase client	firebase_core, firebase_auth, cloud_functions, firebase_app_check, firebase_crashlytics

Layer	Choice
Backend	Cloud Functions 2nd gen, Node.js LTS, TypeScript, region asia-south1 (Mumbai)
AI	Google Gen AI SDK for Gemini; OpenAI-compatible HTTP client for xAI
Build	Two flavors, dev and prod, each with its own Firebase project
Font	Bricolage Grotesque bundled as an asset (not downloaded at runtime)

4.3 Architecture decisions

ID	Decision	Why
AD-01	Flutter, Android first	One codebase, iOS later from the same code
AD-02	Firebase Auth, email and password; Google sign-in in V1.1	No auth server to build or secure
AD-03	Two callable functions plus one auth trigger	The app must show scans left before the first scan of the day, which one function cannot do
AD-04	AI behind a provider adapter: Gemini primary, Grok fallback; model names in config	Swap or upgrade models without an app release; survive one provider's outage
AD-05	Use only the paid tier of each provider	Free-tier inputs may be used for product improvement; user photos must not be
AD-06	History in SQLite on the phone, one file per user id	Privacy, offline use, zero server storage cost
AD-07	Server stores only counters, auto-deleted after 7 days by a TTL policy	Nothing personal to leak or delete later
AD-08	Quota uses reserve-then-refund (section 6.2)	A check-then-count design lets parallel requests beat the limit
AD-09	Scan day is IST on the server; calorie day is the phone's local day	The phone clock cannot be used to gain scans; logs follow what the user lives

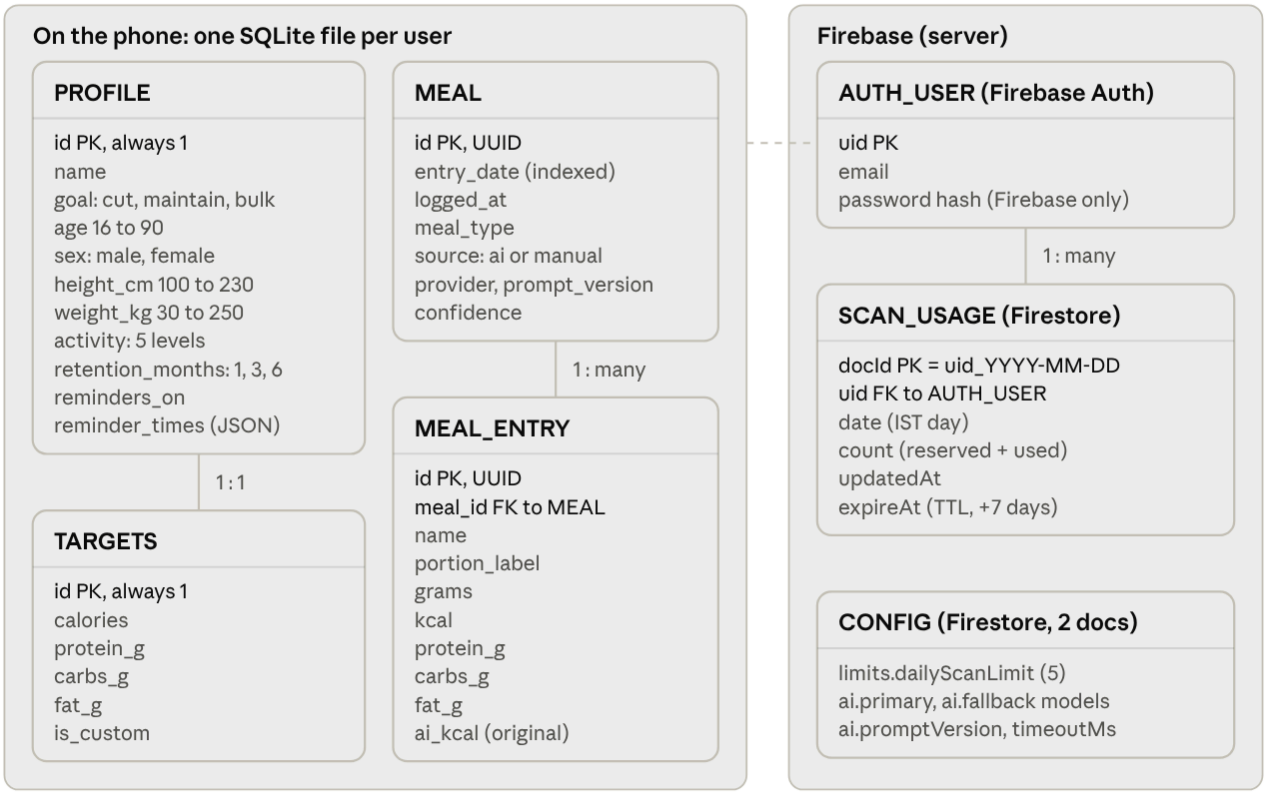
ID	Decision	Why
AD-10	App Check enforced on every callable	Stops scripts from spending the AI budget
AD-11	App depends on a NutritionEstimator interface, not on the cloud call directly	V3 can add an on-device model without touching screens

5. Data design

The phone holds four tables in one SQLite file per user; the server holds only a per-day scan counter and two config documents. A new MEAL table groups the items saved from one scan, and the old alert_log table is removed because V1 no longer sends threshold notifications.

History lives in one SQLite file per user; the server keeps only counters

Entity relationship diagram, Version 1 (PK primary key, FK foreign key)



Dashed: the phone's database file is named after the uid. No meal data is linked to the uid on the server.

ER diagram · 4 local tables, 3 server records

5.1 Local tables (Drift, schemaVersion 1)

Table	Field	Type	Rule
profile	id	int	Always 1
profile	name	text	From sign-up, editable
profile	goal	text	cut, maintain, bulk
profile	age	int	16 to 90
profile	sex	text	male, female; labelled "sex used for the calorie formula"
profile	height_cm, weight_kg	real	100 to 230; 30 to 250
profile	activity	text	sedentary, light, moderate, active, very_active
profile	retention_months	int	1, 3 or 6; default 3
profile	reminders_on	bool	Default true
profile	reminder_times	text	JSON list of HH:MM; default ["13:30", "20:30"]
targets	id	int	Always 1
targets	calories, protein_g, carbs_g, fat_g	int	Whole numbers
targets	is_custom	bool	True after any manual edit; Reset sets it false
meal	id	text	UUID v4, made on the phone
meal	entry_date	text	Local date YYYY-MM-DD; indexed
meal	logged_at	text	ISO 8601 with offset
meal	meal_type	text	breakfast, lunch, snack, dinner; pre-picked from the time of day
meal	source	text	ai or manual
meal	provider, prompt_version	text	Null for manual meals
meal	confidence	text	low, medium, high; null for manual
meal_entry	id	text	UUID v4
meal_entry	meal_id	text	FK to meal.id, ON DELETE CASCADE

Table	Field	Type	Rule
meal_entry	name	text	1 to 60 characters
meal_entry	portion_label	text	Household unit, e.g. "2 roti", "1 katori"; optional
meal_entry	grams	real	Above 0
meal_entry	kcal, protein_g, carbs_g, fat_g	real	0 or more; copied into the row
meal_entry	ai_kcal	real	The model's original kcal; null for manual; used to measure how often users correct the AI

5.2 Data rules

- **Totals are computed, never stored:** `SELECT m.entry_date, SUM(e.kcal), SUM(e.protein_g), SUM(e.carbs_g), SUM(e.fat_g) FROM meal m JOIN meal_entry e ON e.meal_id = m.id GROUP BY m.entry_date`.
- **Nutrition is copied** into each entry, so history never changes if targets or models change.
- **Saving is atomic:** a meal and all its entries are written in one transaction.
- **Deleting the last entry of a meal deletes the meal.**
- **Database file name** is `tracker_<uid>.sqlite`; logout keeps the file; delete account removes it.
- **Backups:** the database is excluded from Android auto-backup in V1 (no opt-in exists yet).
- **Schema changes** go through Drift migrations with a test per version bump.

5.3 Retention rule

On every app start, on resume after the date changes, and right after the setting changes: cutoff = today's local date minus retention_months calendar months (for example, 28 Oct with 1 month gives 28 Sep). Meals with entry_date before the cutoff are deleted; the cutoff day itself is kept. Shortening the period shows a confirmation with the number of meals that will be deleted. Lengthening it never restores anything.

5.4 Server records

Record	Fields	Written by	Read by
scanUsage/{uid}_{date}	uid, date (IST), count, updatedAt, expireAt	analyzeMeal	analyzeMeal, getScanQuota
config/limits	dailyScanLimit (default 5)	Developers in the console	Both functions
config/ai	primary {provider, model}, fallback {provider, model}, promptVersion, timeoutMs (default 12000)	Developers in the console	analyzeMeal

A Firestore TTL policy on expireAt deletes counters 7 days after their day. Security rules deny every client read and write.

6. Interface design

The app calls two Firebase callable functions; a third function runs on its own when an account is deleted. JSON uses camelCase, Dart uses camelCase, SQLite uses snake_case.

6.1 getScanQuota

Called when Home or Camera opens and after the app resumes.

```
// request
{}
// response
{ "limit": 5, "used": 2, "remaining": 3, "resetsAt": "2026-10-
29T00:00:00+05:30" }
```

The app caches the last answer with its resetsAt. Offline, it shows the cached value, or the full limit once resetsAt has passed.

6.2 analyzeMeal

```
// request
{ "imageBase64": "<JPEG, long side 1024 px>", "mealType": "lunch" }

// success response
{
  "scansRemaining": 2,
  "confidence": "medium",
  "provider": "gemini",
  "promptVersion": "p1",
  "items": [
    { "name": "Roti", "portionLabel": "2 roti", "grams": 80, "kcal": 240,
"proteinG": 8, "carbsG": 46, "fatG": 3 },
    { "name": "Dal tadka", "portionLabel": "1 katori", "grams": 150, "kcal":
180, "proteinG": 10, "carbsG": 24, "fatG": 5 }
  ]
}
```

Steps, in order:

1. Reject if there is no signed-in user or no valid App Check token.
2. Reject with BAD_IMAGE if the image is missing, is not a JPEG, or decodes to more than 1 MB.
3. **Reserve:** in a Firestore transaction, read today's counter (IST). If count is at or above the limit, reject with LIMIT_REACHED; otherwise add 1 and set expireAt.
4. Call the primary provider with the image and the fixed prompt for promptVersion. On error, timeout or unusable JSON, call the fallback provider once if at least 6 s of the 20 s budget remain.
5. Validate: schema matches; 1 to 10 items; each item grams 1 to 2,000, kcal 0 to 2,000, macros 0 or more; items failing a range check are dropped. No food found returns NO_FOOD_DETECTED.
6. **Refund:** if no valid result came back, subtract the reserved 1 in a second transaction, then return the error. A failed analysis never uses a scan.
7. Return the result. The image is discarded; logs hold only provider, latency, outcome and item count.

6.3 onUserDeleted

An Authentication trigger that deletes every scanUsage document for that uid. The app deletes the local database file itself before calling the account deletion.

6.4 Error contract

App code	Server sends	Cause	What the app shows
UNAUTHENTICATED	unauthenticated	Signed out, token or App Check invalid	Log in screen; local data kept
BAD_IMAGE	invalid-argument	Missing, wrong type or too large	Retake photo
LIMIT_REACHED	resource-exhausted	Quota used up	Resets at midnight IST; Add manually
NO_FOOD_DETECTED	failed-precondition, reason NO_FOOD	Model found no food	Retake photo; scan not counted
AI_UNAVAILABLE	unavailable or deadline-exceeded	Both providers failed or timed out	Retry and Add manually; scan not counted
NETWORK	(client only)	No connection or client timeout of 25 s	Retry and Add manually

6.5 AI provider adapter

The function depends on one interface, `VisionProvider.analyze(imageBytes, promptVersion, timeoutMs)`, with two implementations. Model names come from config/ai, so a model can be changed without a deploy.

Role	Provider	How it is called	Candidate models (Oct 2026, confirm before Sprint 3)
Primary	Google Gemini API, paid tier	Google Gen AI SDK, JSON response schema, minimal thinking	A current Flash or Flash-Lite model, e.g. gemini-3.8-flash at $0.75 / $3.75 per million input / output tokens (introductory price to 31 Dec 2026) (pricing)
Fallback	xAI Grok API	OpenAI-compatible chat completions, image as data URI, JSON schema output	grok-4.3, vision-capable, $1.25 / $2.50 per million tokens (pricing)

Rules for both: paid tier only (Gemini's free tier uses inputs to improve Google products; the paid tier does not, per Google's billing FAQ); no user id, email or history is ever sent; the final primary and fallback are chosen by the 30-meal accuracy test (section 13), not by preference. If the team meant **Groq** (the fast-inference host) rather than xAI **Grok**, the adapter is the same and only the second implementation changes.

Prompt contract (promptVersion p1): identify each distinct food on the plate; prefer Indian dish names and household portions (roti, katori, bowl, piece); estimate grams; assume typical home-cooking oil unless visible; return JSON only in the schema; return an empty list if there is no food.

Cost estimate: at about 1,500 input and 400 output tokens per scan, one scan costs roughly $0.003 at the Gemini price above, so a user who uses all 5 daily scans costs about $0.40 a month. This is a planning estimate; measure the real token counts in Sprint 3.

7. Module design

The app has four layers and each calls only the one below it: UI, then Logic, then Repositories, then the database or Firebase. Folders are split by feature inside each layer so the two builders rarely touch the same file.

7.1 Folder layout

```
lib/
  core/        theme, constants, date helpers (local day, IST day), error
types, flavors
  ui/          auth/ onboarding/ home/ scan/ history/ settings/   (screens
and widgets only)
  logic/       target_calculator.dart, totals_service.dart,
alert_evaluator.dart,
               portion_scaler.dart, retention_cleaner.dart,
reminder_scheduler.dart
  data/        db/ (Drift database and DAOs), auth/, profile/, meals/, scan/
  estimator/   nutrition_estimator.dart (interface), cloud_estimator.dart
(V1)
functions/
  src/         analyzeMeal.ts, getScanQuota.ts, onUserDeleted.ts,
               quota.ts, providers/gemini.ts, providers/xai.ts, validate.ts,
prompts/p1.ts
```

7.2 Components

Component	Layer	Responsibility	Key inputs and outputs
TargetCalculator	Logic	Profile to calorie and macro targets (section 8.1)	Profile in, Targets out
TotalsService	Logic	Sums entries for a day or a range; fills empty days with zero	Entries in, DailyTotal list out
AlertEvaluator	Logic	Totals and targets to a level	normal, warning or over
PortionScaler	Logic	Rescales an item when grams change	FoodItem, new grams in, FoodItem out
RetentionCleaner	Logic	Deletes meals older than the cutoff	Retention months, today
ReminderScheduler	Logic	Schedules or cancels daily logging reminders	Reminder times, today's meals
AuthRepository	Data	Sign up, log in, log out, reset, delete account, auth state	Firebase Auth

Component	Layer	Responsibility	Key inputs and outputs
ProfileRepository	Data	Read and write profile and targets	Drift
MealRepository	Data	Save meal with entries, update, delete, watch a day, watch totals for a range, delete all	Drift
ScanRepository	Data	Compress, call analyzeMeal and getScanQuota, map errors	Cloud Functions
NutritionEstimator	Estimator	The seam between screens and any AI source	Image in, ScanResult out

7.3 Dart contracts

Type	Fields or methods
FoodItem	name, portionLabel, grams, kcal, proteinG, carbsG, fatG, aiKcal
ScanResult	items, scansRemaining, confidence, provider, promptVersion
ScanQuota	limit, used, remaining, resetsAt
DailyTotal	date, kcal, proteinG, carbsG, fatG, entryCount
Targets	calories, proteinG, carbsG, fatG, isCustom
ScanException	code: UNAUTHENTICATED, BAD_IMAGE, LIMIT_REACHED, NO_FOOD_DETECTED, AI_UNAVAILABLE, NETWORK
AuthRepository	signUp(name, email, password), logIn(email, password), logOut(), sendPasswordReset(email), deleteAccount(password), authStateChanges()
ProfileRepository	getProfile(), saveProfile(p), getTargets(), saveTargets(t), resetTargets()
MealRepository	saveMeal(meal, items), updateEntry(e), deleteEntry(id), deleteMeal(id), watchMealsForDate(d), watchDailyTotals(from, to), deleteOlderThan(date), deleteAll()

Type	Fields or methods
ScanRepository	getQuota(), analyze(file, mealType) returns ScanResult or throws ScanException
NutritionEstimator	estimate(file, mealType) returns ScanResult; V1 implementation delegates to ScanRepository

Every repository has a fake implementation with sample data, so UI work never waits for Firebase.

8. Business rules and algorithms

All rules below run on the phone in plain Dart and each has a unit test. Rounding happens once, at the step named, so every builder gets the same numbers.

8.1 Daily targets

Basal metabolic rate uses Mifflin-St Jeor:

```
BMR = 10 \cdot w + 6.25 \cdot h - 5 \cdot a + s, \quad s = +5 \text{ (male)}, \; -161 \text{ (female)}
```

1. Calories = round(BMR × activity factor × goal factor). Activity: sedentary 1.2, light 1.375, moderate 1.55, active 1.725, very active 1.9. Goal: cut 0.85, maintain 1.0, bulk 1.10.
2. Protein g = round(1.8 × weight kg).
3. Fat g = round(0.25 × calories / 9).
4. Carbs g = round((calories − 4 × protein − 9 × fat) / 4), using the rounded protein and fat; never below 0.
5. Safety floor: the recommendation is never below 1,200 kcal (female) or 1,500 kcal (male); the user may still set a lower custom value after a warning.

Worked example (TC-01): male, 21, 175 cm, 70 kg, moderate, maintain. BMR = 700 + 1,093.75 − 105 + 5 = 1,693.75. Calories = round(2,625.3) = 2,625. Protein 126 g, fat round(72.9) = 73 g, carbs (2,625 − 504 − 657) / 4 = 366 g.

8.2 Custom targets

- Each value can be edited on its own; any edit sets is_custom to true.
- If the macro calories (4P + 4C + 9F) differ from the calorie target by more than 10%, show a warning but allow saving.

- When the profile changes and is_custom is false, targets are recalculated and saved silently. When is_custom is true, the app asks "Recalculate targets from your new details?"
- Reset to recommended recalculates and sets is_custom to false.

8.3 Calorie warning levels

Level	Condition (eaten ÷ target)	Home shows
Normal	below 80%	Progress ring in green
Warning	80% to 100% inclusive	Amber banner: "You have used 85% of today's calories. Keep the next meal light."
Over	above 100%	Red state: "You are 120 kcal over today's target."

The banner is derived from the current totals every time Home draws, so it appears and disappears with edits. Colour is always paired with text and an icon.

8.4 Portion scaling

When an item changes from g0 to g1 grams, kcal, protein, carbs and fat are each multiplied by g1 / g0. Stored values keep one decimal; the screen shows whole numbers. Halving 180 g to 90 g halves every value (TC-05).

8.5 Default meal type

Picked from the phone time, changeable on the Result screen: before 11:00 breakfast, 11:00 to 15:59 lunch, 16:00 to 18:59 snack, from 19:00 dinner.

8.6 History chart

- One bar per calendar day with entries; days with no entries show no bar and are left out of averages.
- Bar colour follows section 8.3 and the target line is drawn across the chart.
- Ranges: 7 days, 30 days, All (the retention window). Above 60 days, bars become weekly averages of logged days, weeks starting Monday.
- Summary under the chart: average kcal per logged day, days over target, days logged out of days in range.

8.7 Logging reminders

Reminders fire at each reminder time (default 13:30 and 20:30). When a meal is saved, today's reminder for that window (the 3 hours before the reminder time) is cancelled, so users who already logged are not nagged. Turning reminders off cancels all of them. Android 13 and later ask for notification permission the first time reminders are enabled.

9. UI and UX design

V1 has 14 screens, three more than v1.1 listed: Forgot password, Manual add and Edit entry were referenced by requirements but missing from the inventory. A scanned meal takes 3 taps from Home: Scan, shutter, Save.

9.1 Screen inventory

ID	Screen	Purpose	States to design
S1	Splash	Route by session and profile	None (under 1 s)
S2	Welcome	Log in or Sign up	
S3	Sign up	Name, email, password with show/hide	Field errors, weak password, email in use, no internet, loading
S4	Log in	Email, password, Forgot password link	Wrong password, no internet, loading
S5	Forgot password	Email field, send reset link	Sent, unknown email shown as sent (no account enumeration)
S6	Profile setup	3 steps: goal; body details; targets review with Edit and Accept; medical note	Invalid ranges disable Continue; custom values marked
S7	Home	Date, calorie ring with kcal left, 3 macro bars, banner, today's meals, scans left, Scan button	Empty day, normal, warning, over, offline
S8	Camera	Preview, shutter, gallery, scans-left chip, Add manually link	Permission rationale, permission denied, limit reached
S9	Result	Thumbnail (memory only), Estimate chip, items with portion and grams, macro chips, stepper, Rescan, meal type, Save	Loading skeleton, success, many items, no food, error with retry

ID	Screen	Purpose	States to design
S10	Manual add	Name, kcal, optional macros, meal type	Validation errors
S11	Edit entry	Change name, grams or values; delete	Delete confirmation
S12	History	Range tabs, bar chart with target line, summary, day list	Empty, fewer days than range, 6-month weekly view
S13	Day detail	Totals vs target, meals and entries	Empty day
S14	Settings	Profile, targets, retention, reminders, medical note, privacy policy, log out, delete data, delete account	Confirm dialogs for every destructive action; re-enter password to delete account

9.2 Navigation

```
Splash -> no session              -> Welcome -> Sign up | Log in (-> Forgot password)
Splash -> session, no profile     -> Profile setup -> Home
Splash -> session and profile     -> Home
Bottom bar: Home | History | [Scan] | Settings
Scan -> Camera -> Result -> Save -> Home
Camera or Result error -> Manual add -> Home
Home or Day detail -> Edit entry
```

9.3 Visual system

Token	Value
Background / card	#F7F9F3 / #EDF1E6
Text / secondary text	#16231A / #4F5A4C
Primary action	#1B7F36 with white text
Protein / carbs / fat	#D9532B / #1C86C4 / #B97A00
Warning / over	#B97A00 / #C62828
Font	Bricolage Grotesque, bundled; system sans fallback

Token	Value
Radius	Cards 24 px, buttons 28 px
Dark theme	Could for V1; follows the system setting when built

The existing mockups (Onboarding, Home, Scan, Result in light and dark) stay valid; Forgot password, Manual add and Edit entry need mockups in Sprint 1.

9.4 Copy and accessibility

- AI numbers are always called "estimate"; manual numbers are not.
- One suggestion per warning, never shaming words.
- Tap targets at least 48 dp; contrast at least 4.5:1; layouts checked at 130% font scale.
- Colour never carries meaning alone: every state has text or an icon.
- Every icon button has a screen-reader label.

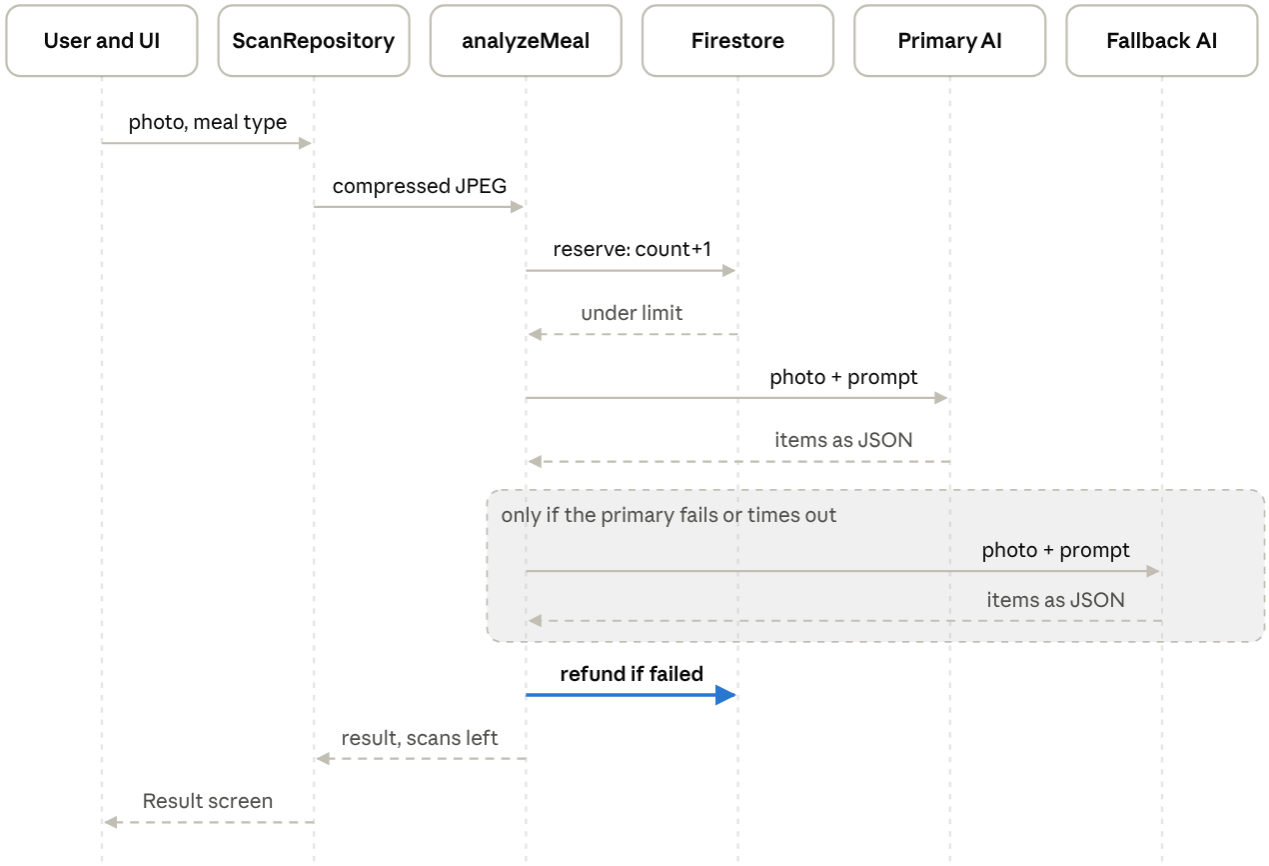
10. Key flows

The scan flow is the heart of V1, and its one non-obvious rule is that the quota is reserved before the AI call and refunded if the call fails.

10.1 Scan a meal

A scan is counted only after the AI returns a valid result

Sequence for one meal scan, with reserve-then-refund quota



Dashed arrows are replies. After Save the app writes the meal to SQLite in one transaction; the photo is never stored.

Scan sequence · 6 participants, fallback and refund

Why reserve first: if the function only counted after success, five scans started at the same moment would all pass the check and a user could get more than the limit. Reserving inside a transaction closes that gap; refunding keeps the promise that failures are free.

10.2 First run

1. Splash finds no session and opens Welcome.
2. Sign up creates the Firebase account; the app creates `tracker_<uid>.sqlite`.
3. Profile setup saves profile and targets; Home opens at zero; the app asks once whether to turn on logging reminders.

10.3 Returning user

1. Splash finds a session and a profile; RetentionCleaner runs; Home opens, even offline.

2. getScanQuota refreshes scans left in the background when online.

10.4 Manual add

1. From Home, Camera or any scan error, the user opens Manual add.
2. Saving writes one meal with source manual; totals and the banner update at once.

10.5 Change retention

1. The user picks a shorter period in Settings; the app shows how many meals will be deleted.
2. On confirm, RetentionCleaner runs and History refreshes.

10.6 Delete account

1. The user taps Delete account and re-enters the password (Firebase requires a recent login).
2. The app deletes the local database file, then deletes the Firebase user.
3. onUserDeleted removes the user's scan counters.
4. The same request can be made without the app through the public deletion page, handled by the team within 30 days.

11. Security, privacy and Play Store compliance

The privacy promise is short enough to put on the store listing: your meals and photos never leave your phone except for the moment a photo is analysed, and nothing about you is sent to the AI provider.

11.1 Controls

Area	Design
Secrets	Provider keys only in Secret Manager, read by the functions; none in the app or the repository
Transport	HTTPS only; Android network security config blocks cleartext
Identity	Firebase ID tokens, verified automatically by callable functions
App integrity	App Check with Play Integrity, enforced on every callable; debug provider only in the dev flavor

Area	Design
Abuse and cost	Per-account daily quota, 1 MB image cap, 20 s function timeout, max instances capped, billing budget alerts
Firestore	Rules deny all client reads and writes
Photos	Held in memory on the server; the phone deletes its temporary file after the call; never logged
Data sent to AI	The image and a fixed prompt only; paid tiers only
Local data	App-private storage; excluded from device backup in V1
Analytics	Firebase Analytics events carry counts and timings only (for example scan_saved with item count and edit count), never food names or nutrition values
Crash reports	Crashlytics; no personal data in custom keys

11.2 Store compliance checklist

Requirement	How V1 meets it
Account deletion	Apps that let users create an account must offer deletion in the app and through a web link entered in Play Console (Play Console Help). V1: Settings, Delete account plus a public deletion page (FR-28).
Data safety form	Declare: email and user id collected; photos sent for processing and not stored; app activity (analytics) collected; data encrypted in transit; deletion available
Privacy policy	Public page on Firebase Hosting, linked in the app and the listing
Health claims	Not-medical-advice note at setup and in Settings; no diagnosis or treatment wording anywhere
Health apps declaration	Complete Play Console's health apps declaration for the fitness and nutrition category; confirm the current form before Sprint 6

Requirement	How V1 meets it
Closed testing	Personal developer accounts created after 13 Nov 2023 need 12 testers opted in for 14 continuous days before production; organisation accounts are exempt (Play Console Help). Publishing under an organisation account avoids this gate.
Notifications	Runtime permission on Android 13 and later, asked only when reminders are turned on

12. Error handling and edge cases

Every failure leaves the user one tap from logging the meal some other way, and no failure loses saved data or costs a scan.

Situation	Behaviour
No internet at scan	NETWORK: Retry and Add manually; no scan used
Both providers fail or time out	AI_UNAVAILABLE: same as above; quota refunded
Photo with no food	NO_FOOD_DETECTED: Retake; quota refunded
Quota used up	Message with the IST reset time; Add manually stays available
Camera permission denied	Explain why, link to system settings; gallery and manual add still work
Camera permission permanently denied	Hide the live preview; show gallery and manual add only
App killed during save	Single transaction: the meal is either fully saved or not at all
App killed during a scan	Function finishes on its own; if the result is valid the scan counts, so the user sees one fewer scan; accepted for V1
Phone date changed by hand	Calorie days follow the phone; quota follows the server, so no scans are gained

Situation	Behaviour
Travel across time zones	Calorie day follows the phone's current zone; entries keep the date they were saved with
Session revoked or password changed elsewhere	Back to Log in; local data kept
Second user logs in on the same phone	Separate database file; first user's data untouched
Profile edited after targets were customised	Ask before recalculating (section 8.2)
Retention shortened	Show count of meals to delete, then delete on confirm
Database migration fails on update	Keep the old file, show an error with a support contact; never delete user data automatically

13. Test design and traceability

28 test cases now cover every Must requirement; v1.1 left 12 requirements without a test. TC-01 to TC-18 keep their old numbers except TC-12, which changed with the notification decision.

13.1 Test levels

Level	Scope	How	When
Unit	Target maths, totals, alert level, portion scaling, retention cutoff, meal-type default, error mapping	Dart tests, run on every pull request	From Sprint 1
Widget	Form validation, Result editing, banner states, chart states	Flutter widget tests	From Sprint 2
Function	Quota reserve and refund, validation, fallback, errors	Firebase emulators with mocked providers, Jest	Sprint 3

Level	Scope	How	When
Integration	Sign up to first meal; scan to saved meal; day rollover; account deletion	integration_test on a device	Sprint 5
Device	Camera, permissions, reminders, performance	Manual on at least 3 phones, Android 10 to 15, one with 3 GB RAM	Sprint 5
AI accuracy	30 known meals (at least 15 Indian home-cooked plates), every candidate model	Spreadsheet: true kcal, estimated kcal, % error, latency, cost	Sprint 3, repeated on any model change
Usability	Time to log a meal; clarity of warnings	5 gym-goers, think-aloud, timed	Sprint 5

13.2 Test cases

ID	Covers	Test	Expected
TC-01	FR-07	Male, 21, 175 cm, 70 kg, moderate, maintain	2,625 kcal, 126 g protein, 73 g fat, 366 g carbs
TC-02	FR-06	Age 5, or weight 0	Inline error, Continue disabled
TC-03	FR-03	Log in, close, reopen in airplane mode	Home opens without login
TC-04	FR-01	Sign up with an email already used	Clear error
TC-05	FR-14	Change an item from 180 g to 90 g	Every value halves
TC-06	FR-15, FR-20	Save a result	Home totals update at once
TC-07	FR-17	Scan up to the limit, then once more	Last call returns LIMIT_REACHED
TC-08	FR-17	Change the phone date, then scan	Quota still enforced
TC-09	FR-12	Scan with internet off	Retry and manual options; scans left unchanged

ID	Covers	Test	Expected
TC-10	FR-25	Target 2,000; log 1,600	Warning state
TC-11	FR-25	Push total to 2,001	Over state
TC-12	FR-26	Reminder at 13:30; log a meal at 12:00	13:30 reminder does not fire
TC-13	FR-19	Move the phone to the next day	New day at zero; old day in history
TC-14	FR-22	Retention 1 month with 2-month-old data	Confirmation with count, then deletion
TC-15	FR-23	Chart with 6 months of data	Weekly averages, scrolls without lag
TC-16	FR-09	Deny camera permission	Explanation, settings link, manual add works
TC-17	FR-11	Photo of a non-food object	NO_FOOD_DETECTED; scan not counted
TC-18	AD-06	Log out, log in as another user	Separate history
TC-19	FR-17, AD-08	Fire 6 scans at once with limit 5	Exactly 5 succeed
TC-20	FR-11	Primary provider mocked to fail	Fallback result returned; one scan used
TC-21	FR-28	Delete account	Local file gone, counters gone, cannot log in
TC-22	FR-04	Request a password reset	Reset email arrives; same message for unknown email
TC-23	FR-16	Manual add with only name and kcal	Saved; totals update
TC-24	FR-21	Edit then delete an entry	Totals update after each step
TC-25	FR-07	Change weight with non-custom targets	Targets recalculate

ID	Covers	Test	Expected
TC-26	NFR-09	Call analyzeMeal from a script without App Check	Rejected
TC-27	FR-18	Open Camera before any scan today	Shows full limit from getScanQuota
TC-28	NFR-03	Use Home, History, manual add offline	All work

13.3 Traceability

Feature	Requirements	Screens	Components	Tests
F1 Account and targets	FR-01 to FR-08, FR-28	S1 to S6, S14	AuthRepository, ProfileRepository, TargetCalculator, onUserDeleted	TC-01 to TC-04, TC-18, TC-21, TC-22, TC-25
F2 Photo analysis	FR-09 to FR-12	S8, S9	ScanRepository, NutritionEstimator, analyzeMeal, providers	TC-09, TC-16, TC-17, TC-20, TC-26
F3 Review and log	FR-13 to FR-16	S9, S10	PortionScaler, MealRepository	TC-05, TC-06, TC-23
F4 Scan quota	FR-17, FR-18	S7, S8	analyzeMeal, getScanQuota, quota.ts	TC-07, TC-08, TC-19, TC-27
F5 Today, history, warnings	FR-19 to FR-27	S7, S11 to S14	TotalsService, AlertEvaluator, RetentionCleaner, ReminderScheduler	TC-10 to TC-15, TC-24, TC-28

13.4 Defect levels and release gate

Level	Meaning	Rule
Critical	Crash, data loss, wrong totals, login broken, camera unusable, privacy leak	Fix before any release
Major	A Must requirement does not work as specified	Fix before release
Minor	Cosmetic or small usability issue	Fix if time allows

V1 may go to the closed test when all Must tests pass, no Critical or Major defect is open, it has run on 3 phones, and most single dishes in the 30-meal set are within 25% on calories.

14. Environments, deployment and operations

Two fully separate Firebase projects and two app flavors mean a test can never touch real users or the real AI budget.

Item	Design
Environments	food-macro-dev and food-macro-prod Firebase projects; app flavors dev and prod, each with its own Firebase config file; the dev flavor shows a DEV badge
Source control	GitHub, protected main branch, one branch per task (feature/<area>-<task>), pull request with review from the other builder
CI	GitHub Actions on every pull request: flutter analyze, flutter test, functions lint and tests against the emulators
Backend deploy	`firebase deploy --only functions,firestore` from main, dev first, prod after the release gate
App build	`flutter build appbundle --release --flavor prod` with code shrinking; upload key kept outside the repository with a backup; Play App Signing on
Config without release	config/limits and config/ai changed in the console; every change noted in the team chat with date and reason
Monitoring	Function logs (no image data), Crashlytics, Analytics dashboards for the metrics in Appendix B
Cost control	Budget alerts on both Firebase projects and on each AI provider account; functions capped at a set number of instances
Versioning	Semantic versions; Git tag per release; versionCode increases every upload

Item	Design
Release path	Internal testing, then closed testing (12 testers for 14 days if publishing from a new personal account), then production

15. Open decisions and risks

Five decisions remain open; each has a default so work never waits.

#	Decision	Default until decided	Decide by
1	Grok (xAI) or Groq as the second provider	xAI Grok	Sprint 3 start
2	Final primary and fallback models	Chosen from the 30-meal test on accuracy, then latency, then cost	End of Sprint 3
3	Daily scan limit	5 per day	After closed test data
4	Publish from a personal or organisation Play account	Organisation account if the team has one (skips the 12-tester gate)	Sprint 5
5	Launch date and first marketing channel	Not decided	Sprint 5

Risk	Likelihood	Impact	Mitigation
AI accuracy too low on Indian plates	Medium	High	30-meal gate, household portion units, easy editing, prompt versions, two providers
AI cost grows faster than users	Low	Medium	Quota, App Check, budget alerts, Flash-class models
Model names deprecated by providers	High	Medium	Model ids in config; check provider deprecation pages each sprint
Users quit in week 1	High	High	Fast logging, reminders, measure D1 and D7 from day one

Risk	Likelihood	Impact	Mitigation
History lost on phone change	Medium	Medium	Clear in-app message; cloud backup in V2
Beginners stuck on setup or Gradle	Medium	Medium	Pair sessions, troubleshooting table, timebox of 1 hour before asking

Appendix A. Discrepancy log

32 issues were found across the three files: 11 contradictions between files, 12 gaps where a requirement had no design, and 9 over-scoped or risky items. Each is resolved in the section shown.

A.1 Contradictions between files

#	Where	Conflict	Resolution	See
1	Design 9.2 vs Guide 2.1	Design gives auth and the backend to Vishal; the Guide gives the Firebase project, auth setup and the limit counter to Riya	One ownership table in Appendix C	C.4
2	Guide 2.1 vs Guide Phase 8	Result screen owned by Vishal in the table, built by Riya in Phase 8	Riya owns Result; Vishal owns Camera and ScanRepository	C.4
3	Guide 2.1 vs Guide Phase 6	Manual add and Edit entry owned by Riya in the table, built by Vishal in Phase 6	Vishal owns both, matching the meals data layer he owns	C.4
4	Design 7.2 vs Design 11.4 vs Guide Phase 6	Banner shown by state in one place, only once per day (via alert_log) in the others	Banner always follows current totals; once-per-day applied only to notifications, which V1 no longer sends	8.3

#	Where	Conflict	Resolution	See
5	Design 8.2 vs Guide Phase 7	"Read count, call LLM, add 1 after success" inside one transaction; a transaction cannot wait on a slow AI call and the check can be beaten by parallel requests	Reserve-then-refund in two short transactions	6.2, 10.1
6	Design D2, FR-05, open point 2	Google sign-in both a V1 Should and "later"	Could, scheduled for V1.1	3.1
7	Design stack vs Guide brief	go_router and "Riverpod without code generation" only in the Guide	Both adopted in the stack	4.2
8	Design 8.2 vs Architecture 3	grams above 0 rule only in the Architecture file; "clamp or reject" undecided	Ranges defined; failing items dropped	6.2
9	User request vs all files	Team plans to use Google and Grok keys; files name Gemini only	Provider adapter with Gemini primary and Grok fallback	6.5
10	Design 12	Local data "excluded from backups unless the user opts in" but no opt-in exists	Excluded in V1, no opt-in	5.2
11	NFR-07 vs Android guidance	44 px tap targets (an iOS figure)	48 dp	3.2

A.2 Gaps: a requirement with no design

#	Gap	Resolution	See
12	FR-18 needs scans left before the first scan, but clients cannot read Firestore and only analyzeMeal returned the count	New getScanQuota callable	6.1

#	Gap	Resolution	See
13	Manual add was a Should, yet every error path depends on it	Made Must	3.1
14	Edit entry, Manual add and Forgot password screens missing from the inventory	Added as S5, S10, S11	9.1
15	Items from one scan saved as unrelated rows; Home lists "meals"	MEAL table groups entries	5.1
16	Where the user picks meal type was not designed	Default by time of day, editable on Result	8.5
17	Profile changes did not say what happens to targets	Recalculate if not custom, ask if custom	8.2
18	Chart averages and empty days undefined	Defined	8.6
19	"Today minus retention_months" ambiguous (30 days or calendar months)	Calendar months; cutoff day kept	5.3
20	The protein reminder needed the app running at reminder time	Moved to V1.1; reminders now pre-scheduled and cancelled on save	8.7
21	Delete account needs a recent login; scan counters "deleted or left to expire"	Re-enter password; auth trigger plus TTL	10.6
22	Switching the app from dev to prod by re-running flutterfire configure overwrites the dev config	Two build flavors	14
23	12 of 28 requirements had no test case	10 new cases; every Must covered	13

A.3 Over-scoped or risky items

#	Item	Why it was a problem	Change
24	Threshold notifications (80% and 100%)	Every crossing happens while the user is in the app, so a notification repeats what the screen already shows; needed a whole alert_log table	Removed; replaced by logging reminders, which bring users back
25	alert_log table	Only served item 24	Removed
26	Account deletion as a Should	Google Play requires in-app and web deletion for apps with accounts	Must, with web page
27	Free AI tiers implied by "set a budget alert" only	Free-tier inputs may be used to improve the provider's products; user photos would be included	Paid tiers only
28	One hard-coded model name	Providers retire models every few months	Model ids in config/ai
29	Gender limited to male and female with no explanation	Reads as exclusionary	Labelled "sex used for the calorie formula"
30	No minimum calorie recommendation	Cut goal for small users could recommend unsafe intake	Floor of 1,200 / 1,500 kcal
31	No App Check in the plan (only a Should)	Anyone with the function URL could spend the AI budget	Must, enforced
32	No way to measure whether the app works for users	No analytics in any file	Count-only analytics events and KPIs

Appendix B. Product strategy

We win the way Blinkit and Zepto beat BigBasket: not with more features, but by owning the one number users feel every single time. For quick commerce that number was minutes to the door. For us it is **seconds to log a meal**, backed by numbers that are right for Indian plates.

B.1 The quick-commerce lesson

BigBasket had the bigger catalogue, the supply chain and the trusted brand, built for planned, scheduled shopping. Blinkit and Zepto attacked a different job, the urgent top-up, and built a new operating model (dark stores near homes) around a 10-minute promise. Tata-owned BigBasket ended up moving into full-scale quick commerce itself (Inc42). Three lessons carry over:

1. **Pick one felt metric and design the whole system around it.** Delivery time for them, logging time for us.
2. **Serve a narrower, more frequent moment.** They served "I need milk now"; we serve "I just ate, log it before I forget".
3. **The incumbent's strengths stop mattering if they do not help that moment.** A huge food database does not help a user who will not type.

B.2 Why calorie apps lose users

The category's problem is not features but drop-off. One study cited by Cordeiro found that only 3% of 190,000 downloads of a food-journal app led to use beyond one week (Big Think). Peer-reviewed work on nutrition apps names the time each entry takes as a main reason people quit early (JMIR mHealth). A competitor's own 2026 data, which should be read with caution as vendor research, puts first-app churn at 65% within 90 days and finds that users who log on 5 or more days in week 1 retain far better (Nutrola). The design consequence: week 1 is everything, and every second removed from logging is retention.

B.3 Competitors

App	What it is	Strength	Gap we can use
HealthifyMe	India's large health platform; Snap photo logging; AI coach Ria; paid plans with coaches	Big Indian food database; Snap reached about 80% accuracy on single Indian foods	Heavy, coaching-led app; photo logging sits behind a paid plan (around ₹2,499 a year per this review); multi-item plates historically weaker
Cal AI	Photo-first tracker, about $29.99 a year; acquired by MyFitnessPal (announced March 2026)	Proved photo-first logging sells	Western food focus; free tier has no AI scanning; trust issues reported in 2026 (data breach, temporary App Store removal over paywall design)

App	What it is	Strength	Gap we can use
MyFitnessPal	Long-running manual and barcode tracker, now also owns Cal AI	Database and brand	Typing-heavy; generic for Indian home food
Many small "AI calorie" clones	Thin wrappers on a general AI model	Cheap to build	Few reach real revenue; most sell for a few thousand dollars on startup marketplaces

Accuracy is a category-wide weakness: an NIH-led test of four photo apps found they tend to underestimate calories, especially from fats (teardown summary). For Indian cooking, where oil and ghee vary meal to meal, that is exactly where we can be better.

B.4 Our wedge: what we do that they do not

Wedge	In V1	Grows into
Fastest log	3 taps from Home; median under 15 s; scan result is editable, never retyped	Quick-capture widget, lock-screen shortcut, live camera estimate (V3)
Indian plate truth	Household units (roti, katori, piece), multi-item thalis, oil assumption in the prompt	One-tap oil level (light, normal, heavy), regional dish names, mess and canteen presets
Protein first	Protein shown as large as calories; aimed at gym-goers and vegetarians who under-eat protein	Protein gap suggestions from common Indian foods
Trust	History stays on the phone; honest "estimate" labels; generous free scans; no dark-pattern paywall	Opt-in cloud backup; transparent accuracy report
Habit, not guilt	Calm warnings, smart reminders that skip if you already logged	Streaks that forgive one missed day, weekly summary

What we deliberately do not build in V1: coaching, diet plans, a giant searchable database, social feeds. Each is something incumbents do better; none makes logging faster.

Positioning line: "Click your thali. Get your macros in seconds. Your food diary stays on your phone."

B.5 Launch first, then improve every two weeks

The team's instinct is right: the product has to exist before it can get better, and real users teach more than any document. The guardrail is that shipping early is about scope, not quality: V1 ships with few features, but never with wrong totals, lost logs or privacy leaks, because those destroy the trust a tracker needs. The release gate in section 13.4 encodes that line.

From the first closed-test build, track these and review them every sprint:

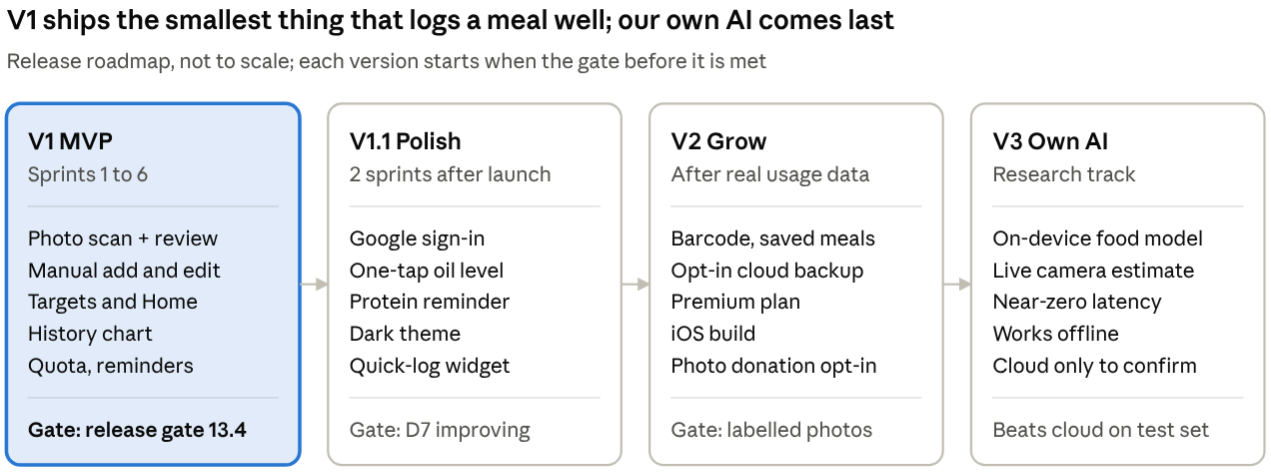
Metric	Definition	First target
Activation	New users who save a meal in their first session	60%
Time to log	Median seconds from app open to meal saved (scan path)	Under 15 s
Week-1 logging	Users who log on 5 or more of their first 7 days	30%
D1 / D7 / D30 retention	Users who log again on day 1, 7, 30	Baseline in closed test, then improve each release
AI edit rate	Share of AI items the user changed before saving	Falling over time
Scan to manual ratio	Share of meals logged by scan	Rising over time
Crash-free sessions	From Crashlytics	Above 99.5%
AI cost per weekly active user	Provider spend ÷ WAU	Within budget

The first targets are starting assumptions for the team to revise after the closed test, not industry benchmarks.

Appendix C. Roadmap and delivery

V1 is planned as six 2-week sprints after this design sprint, aiming at a closed test in late January 2027. The dates are targets that assume work starts on Monday 2 November 2026 and should be re-planned at every sprint review.

C.1 Roadmap



Roadmap · V1 to V3 with gates

The on-device model is last for a reason: it needs labelled meal photos, and V1's privacy design stores none. V2 therefore adds an opt-in photo donation programme, alongside public food-image datasets, to build the training set.

C.2 Sprint plan for V1

Sprint	Target dates	Goal	Done when
0	26 Oct to 1 Nov	Design baseline (this document), tools installed, repository set up	Both builders run the default app on a phone and push to the repo
1	2 to 15 Nov	Skeleton, theme, navigation, contracts with fakes, Firebase dev project and flavors, local database, logic with tests	TC-01, TC-05 pass; all 14 screens open as placeholders
2	16 to 29 Nov	Sign up, login, forgot password, profile setup, Home, manual add, edit and delete	TC-02 to TC-04, TC-06, TC-10, TC-11, TC-22 to TC-25 pass
3	30 Nov to 13 Dec	Functions: quota, providers, validation, emulator tests; 30-meal accuracy test; pick models	TC-07, TC-08, TC-17, TC-19, TC-20, TC-26, TC-27 pass on the emulator
4	14 to 27 Dec	Camera, Result, real scan end to end; history chart, day detail, retention	A real photo becomes a saved meal; TC-09, TC-13 to TC-16 pass

Sprint	Target dates	Goal	Done when
5	28 Dec to 10 Jan	Settings, reminders, account deletion, analytics, App Check; device and usability tests	TC-12, TC-18, TC-21, TC-28 pass; tested on 3 phones
6	11 to 24 Jan	Bug fixing, prod project, store listing, privacy and deletion pages, release build	Release gate in 13.4 met; build in closed testing
Closed test	25 Jan to 7 Feb	12 or more testers for 14 days; fix what they find	Production access granted

C.3 Agile routine

When	What
Sprint start (Monday)	Planning: pick stories from the backlog, each with acceptance criteria and test IDs
Daily	Two-line update in the team chat: done yesterday, doing today, blocked by
Twice a week	30-minute call: review each other's pull requests, clear blockers
Sprint end (second Friday)	Demo on a real phone, then a 15-minute retrospective: keep, stop, try
Every sprint	Update this document's version if a Must requirement, the data model or an API changed

C.4 Ownership

Each builder owns whole vertical slices, so both learn frontend, backend, database and testing. The owner writes the code; the other reviews it.

Area	Vishal	Riya
Screens	Splash, Welcome, Sign up, Log in, Forgot password, Camera, Manual add, Edit entry	Profile setup, Home, Result, History, Day detail, Settings

Area	Vishal	Riya
Logic	PortionScaler, RetentionCleaner, meal-type default, error mapping	TargetCalculator, TotalsService, AlertEvaluator, ReminderScheduler
Data	AuthRepository, MealRepository, ScanRepository, meal and meal_entry tables	ProfileRepository, profile and targets tables, database setup and migrations
Backend	Provider adapter, prompts, validation, analyzeMeal	Firebase projects, rules, App Check, quota module, getScanQuota, onUserDeleted, config documents
Quality	Function tests, usability test with 5 gym-goers	30-meal accuracy test, integration tests
Release	Signing key, prod build, CI pipeline	Privacy policy, deletion page, store listing, Data safety form
Together (pair)	Sprint 1 skeleton, first function deploy, end-to-end scan in Sprint 4, device testing	

A file in the other person's area is changed only through a pull request the owner approves.

C.5 Definition of Done

- [ ] Meets the requirement's acceptance criteria on a real phone
- [ ] Logic has unit tests; linked TC IDs pass
- [ ] flutter analyze and CI are clean
- [ ] No calculation in UI code, no direct database or Firebase call from UI
- [ ] No secrets, personal data or photos in code or logs
- [ ] Reviewed and merged by the other builder, who can explain it
- [ ] This document updated if the design changed

C.6 Path to the on-device live model

Step	Version	What
Seam	V1	Screens depend on NutritionEstimator, not on the cloud call (AD-11)

Step	Version	What
Data	V2	Opt-in photo donation with the user's corrected values as labels; public food-image datasets; clear consent and deletion
Model	V3	Small detection and portion model exported to an on-device runtime (LiteRT / TensorFlow Lite) and run on camera frames
Hybrid	V3	Live on-device estimate while framing; cloud provider confirms on shutter until the local model wins on the 30-meal test
Switch	V3+	Local model becomes primary when it matches cloud accuracy, cutting latency and AI cost

C.7 Working with AI assistants

- Start each AI chat with the project brief: stack, layer rules, "meal data stays on the phone", "keys only in Secret Manager".
- Ask for one small piece at a time (one screen, one function, one test) with file names and tests.
- Run it at once; paste the exact error and file when it fails.
- Never paste keys, passwords or signing files into a chat.
- If the AI disagrees with this document, follow the document and raise a design change if the AI is right.
- Check package versions against the official package page; AI knowledge of versions goes stale.

Sources

Prices, policies and model names were checked on 28 October 2026 and change often; re-check before Sprint 3.

- Gemini API pricing, October 2026
- Gemini Developer API pricing (Google)
- Gemini API billing and paid-tier data use (Google)
- Gemini deprecations (Google)
- Grok API pricing 2026 (CometAPI)
- Account deletion requirements (Play Console Help)
- Testing requirements for new personal developer accounts (Play Console Help)

- Firestore TTL policies (Firebase)
- Zepto bulk orders and BigBasket's quick-commerce move (Inc42)
- Why people quit food logging apps (Big Think)
- Nutrition app use and early quitting (JMIR mHealth)
- First-app churn report 2026 (Nutrola, vendor data)
- Healthify customer story (OpenAI)
- HealthifyMe 2026 review (NutriScan)
- Cal AI pricing 2026 (eesel)
- Cal AI product teardown (RaftLabs)